

보안 project1

간단한 로그인 프로그램 리버싱해서 인증 우회 원리 분석하기

정명훈

개요:이 프로젝트의 목표는 리버싱의 기본적인 원리와 어떻게 리버싱을 하는지에 대한 과정의 실제 수행에 있고 보고서의 간결함을 위해 proj1,3에 대한 분석 내용에 대한 내용은 생략하고 proj2에 대한 분석만 작성했고 이 프로젝트에 나오는 모든 프로그램은 상용 프로그램이 아닌 프로젝트를 위해 만든 임의의 프로그램임을 알린다.

환경: ida9.3, visual studio2022,ghidra_12.1.2_PUBLIC_20260605,윈도우 환경에서 실행했습니다.

1일차에는 Ghidra,x64dbg,Detect It Easy,HxD,GCC 등의 프로그램을 깔고 PE 파일 구조 이해 문자열 분석 함수 목록 보기 디스어셈블리와 디컴파일 차이 이해에 대한 기본적인 개념을 학습했다. 우선 PE(Portable Executable)는 Windows 실행 파일(EXE, DLL)의 표준 형식으로. 실행 파일은 용도에 따라 여러 영역(Section)으로 나뉘어 저장된다. 주요 영역은

섹션	역할
.text	실제 프로그램 코드(어셈블리 명령어)
.rdata	읽기 전용 데이터(문자열, 상수)
.data	전역 변수 및 초기화된 데이터
.rsrc	아이콘, 이미지, 메뉴 등의 리소스
.pdata	예외 처리 정보(x64)

다음과 같다.

문자열 분석은 프로그램의 기능을 가장 빠르게 파악하는 방법이다.예를 들어 다음과 같은 문자

열 Enter password,Access granted,Access denied가 존재한다면 로그인 기능이 존재한다는 것을 쉽게 추측할 수 있다. Ghidra에서는 Defined Strings 창에서 문자열을 확인한 후, XREF(Cross Reference) 를 이용하여 해당 문자열을 사용하는 함수를 찾을 수 있다. xref방법은

Ghidra기준으로 windows -> defined strings -> 찾으려 하는 문자열을 입력 -> 그 다음에 거기 로 이동한 다음에 우클릭 후 references -> show references to address에 가서 그 문자열이 사용되는 함수를 찾을 수 있다.

함수(Function)는 프로그램의 기능을 수행하는 코드 단위다.

Ghidra의 Symbol Tree → Functions 에서는 프로그램에 존재하는 함수 목록을 확인할 수 있다. 예를 들어 main,check_password,printf,scanf,strcmp 와 같은 함수가 있다면 프로그램의 전체 동작 흐름을 쉽게 이해할 수 있습니다. 실제 악성코드나 상용 프로그램에서는 함수명이 제거되어 FUN_1400010e0, FUN_140001250와 같이 표시되는 경우가 많다.이때는 문자열과 XREF를 이용하여 함수의 역할을 추론한다.

또한 디스어셈블리어와 디컴파일의 차이를 요약하면 아래의 표와 같다

디스어셈블리	디컴파일
CPU가 실제 실행하는 코드	사람이 이해하기 쉬운 C 형태
정확도가 가장 높음	분석이 쉬움
수정(패치) 시 주로 사용	프로그램의 동작을 이해할 때 사용

2일차에는 실제로 분석할 실행파일을 3개 만들기 위해 간단한 로그인 프로그램을 세 개의 버전으로 만들어 실행파일을 만들었다 하나는 그냥 문자열을 하나는 xor을 이용한 인코딩된 문자열을 나머지 하나는 다양한 조건과 함수를 사용하는 방식으로 만들었다. 그 다음에는 ghirda에 exe파일을 올리고 실제로 1일차에 배운 내용을 기반으로 분석을 진행해 보았다.



우선은 먼저 xref를 연습해 보기 위해 문자열 테이블을 보고 그 중에서 의미가 있는 걸 찾아보는 중 access denied, access granted가 눈에 띄어 거기를 누르고 거기서 우클릭 후 reference -> show reference address를 누르니 그 문자열이 쓰인 함수로 같은데

```

01
LAB_140001178 XREF[1]:
main.c:33 (28)
140001178 45 85 c0 TEST R8D,R8D
14000117b 48 8d 05 LEA RAX,[s_Access_denied_]
0e 21 00
00
140001182 48 8d 0d LEA RCX,[s_Access_granted_]
f7 20 00
00
140001189 48 0f 45 CMOVNZ RCX=>s_Access_denied_,RAX
c8
14000118d e8 8e fe CALL printf
ff ff
140001192 33 c0 XOR EAX,EAX
main.c:34 (18)

```

그곳은 main함수였고 이 파일의 경우 main함수가 이름이 드러난 간단한 파일이라 symbol tree의 function부분에서도 쉽게 이를 확인할 수 있었다 그 다음에는 anlyaze를 통해서 디컴파일된 내용을 보니

<pre> 140001163 3a 0c 10 CMP CL,byte ptr [RAX + RDX*0x1] 140001166 75 09 JNZ LAB_140001171 140001168 48 ff c0 INC RAX 14000116b 84 c9 TEST CL,CL 14000116d 75 f1 JNZ LAB_140001160 14000116f eb 07 JMP LAB_140001178 LAB_140001171 XREF[1]: 140001171 45 1b c0 SBB R8D,R8D 140001174 41 83 c8 OR R8D,0x1 01 LAB_140001178 XREF[1]: main.c:33 (28) 140001178 45 85 c0 TEST R8D,R8D 14000117b 48 8d 05 LEA RAX,[s_Access_denied_] 0e 21 00 00 140001182 48 8d 0d LEA RCX,[s_Access_granted_] f7 20 00 00 140001189 48 0f 45 CMOVNZ RCX=>s_Access_denied_,RAX </pre>	<pre> 32 printf("Enter password: "); 33 scanf("%31s"); 34 lVar2 = 0; 35 uVar5 = 0; 36 do { 37 local_48[lVar2] = local_48[lVar2] ^ 0x 38 lVar2 = lVar2 + 1; 39 } while (lVar2 < 0xd); 40 pbVar3 = local_38; 41 lVar2 = -(longlong)pbVar3; 42 do { 43 bVar1 = *pbVar3; 44 if (bVar1 != pbVar3[(longlong)(local_4 45 uVar5 = -(uint)(bVar1 < pbVar3[(long 46 break; 47 } 48 pbVar3 = pbVar3 + 1; 49 } while (bVar1 != 0); 50 pcVar4 = "Access granted "; 51 if (uVar5 != 0) { 52 pcVar4 = "Access denied\n"; 53 } </pre>
--	---

이런 형태로 나왔고 분기 부분의 어셈블리를 확인하고 또 디컴파일된 코드의 내용을 본인이 한 번 자세히 추론해 보니 우선 성공하기 위해서는 uvar5 == 0이 되어야 하는 거 같다는 걸 알아냈고 그 위의 while문은 uVar5 = strcmp(local_38, local_48);을 나타내며 local_48[lVar2] = local_48[lVar2] ^ 0x55; 은 xor을 이용한 복호화 과정이고 맨 위의 local48배열은 암호화된 비밀번호 배열같았고

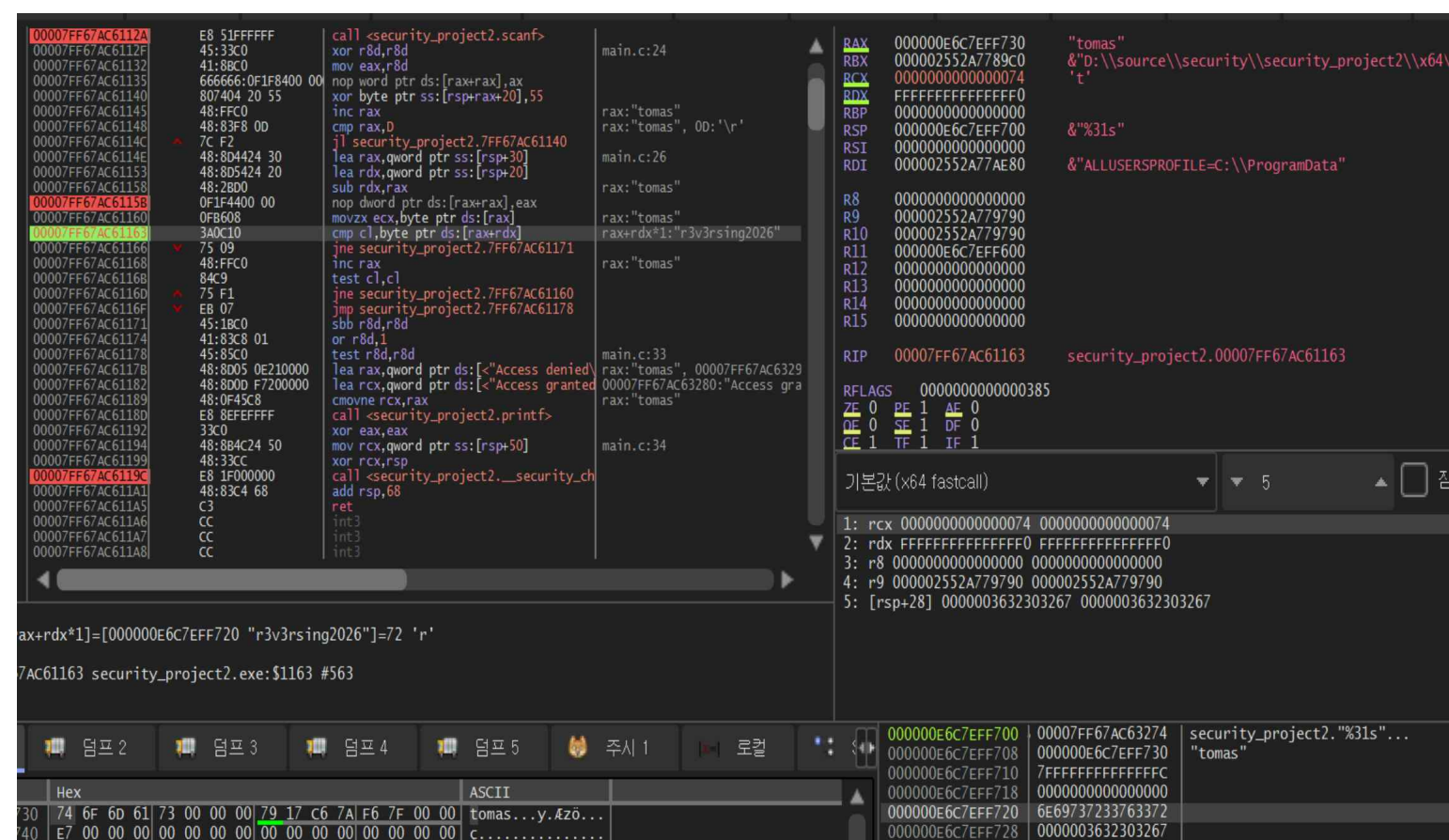
<pre> main.c:22 (17) 14000111e 48 8d 54 LEA RDX=>local_38,[RSP + 0x30] 24 30 140001123 48 8d 0d LEA RCX,[s_%31s] 4a 21 00 00 14000112a e8 51 ff CALL scanf ff ff </pre>	<pre> 27 local_48[9] = 0x67; 28 local_48[10] = 0x65; 29 local_48[0xb] = 0x67; 30 local_48[0xc] = 99; 31 local_48[0xd] = 0; 32 printf("Enter password: "); 33 scanf("%31s"); 34 lVar2 = 0; </pre>
--	--

아래의 스크린샷을 보면 scanf의 인자가 디컴파일에서는 안보이지만 어셈블리어의 rdx가 local_38인 것을 보아서 실제로는 local38값을 입력받는 다는 것을 알 수 있고 이를 local48의 복호화된 값과 같은지 비교해서 맞으면 access granted가 뜨는 flag check문제라는 사실을 알 수 있다.

그 다음에 이렇게 정적분석을 한 후에는 동적 분석을 위해 이 프로그램을 xdbg64에서 실행해 보았다(사실 이 프로그램은 너무나 간단해서 정적 분석만으로도 충분하지만 연습을 위해서 했다.) 우선 먼저 f9를 눌러서 실행을 해봤다 실행결과를 알기 위해서는 xdbg에서도 마찬가지로 xref로 문자열 access denied부분을 찾아서 break point를 설정한 다음에 잘못된 입력을 넣어서 결과를 확인하니



위와 같은 결과가 나왔다. 그 다음에는 strcmp주변의 값들을 디버깅을 통해서 분석을 해 보았는데

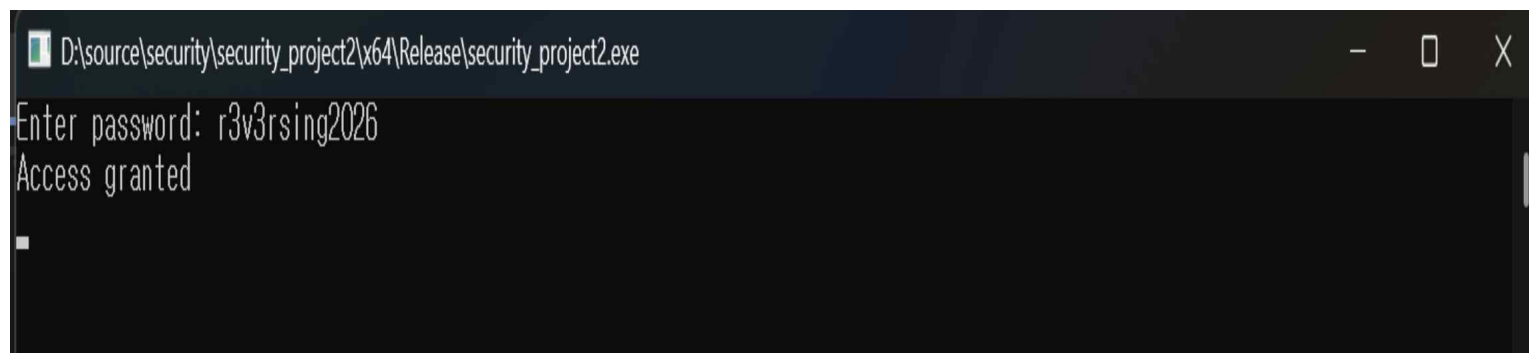


화면이 너무 작아서 잘 안보이겠지만 우선 scanf를 ida와 xdbg64의 주소를 맞추고 ida의 주소를 이용해 xdbg64에서 주소를 찾은 다음에 breakpoint를 설정해 입력창에 tomas를 치고 레지스터에 tomas가 적힌걸 확인했다. 그 다음에는 다음 breakpoint를 xor후 복호화된 값으로 잡고 했더니

주소	Hex	ASCII
000000E6C7EFF720	72 33 76 33 72 73 69 6E 67 32 30 32 36 00 00 00	r3v3rsing2026...

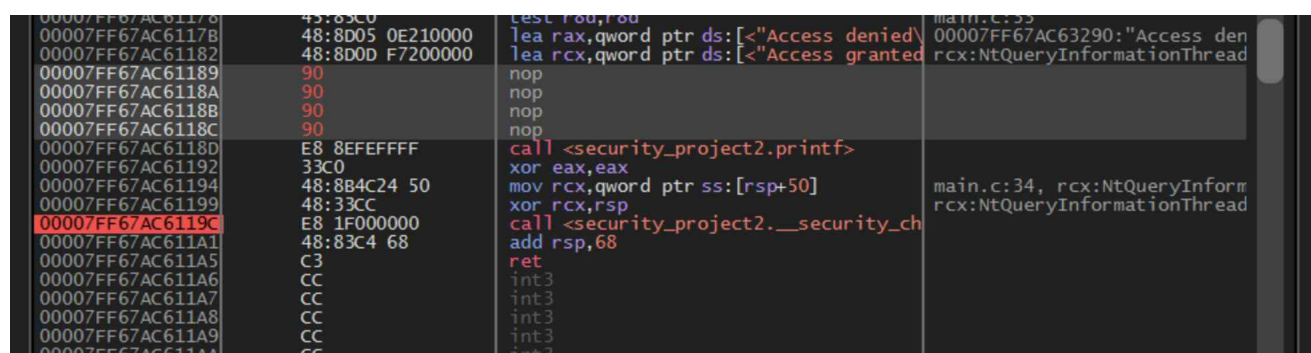
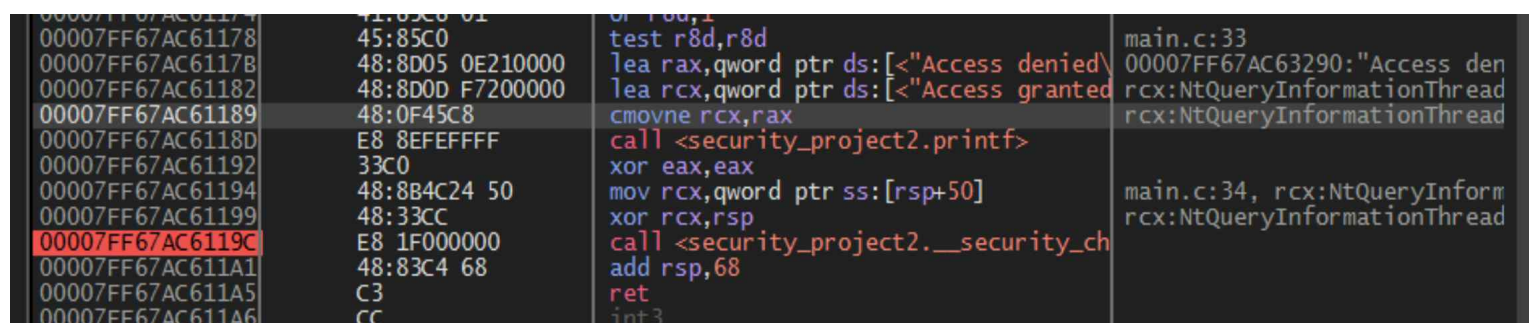
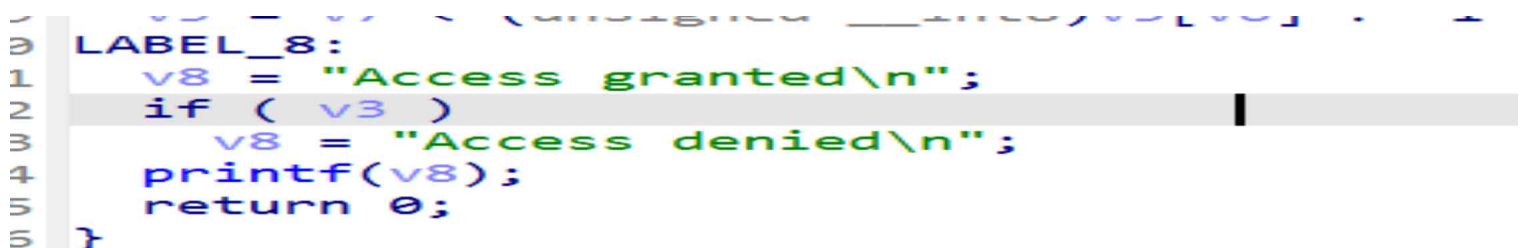
아래처럼 복호화된 값이 나와 쉽게 문제의 정답을 찾을 수 있었다. 그 다음에는 내가 입력한 tomas와 r3v3rsing2026이 다르기 때문에 정답이 아니라 denied가 될 거라는 걸 알 수 있음

로 그 이후 디버깅은 멈추고

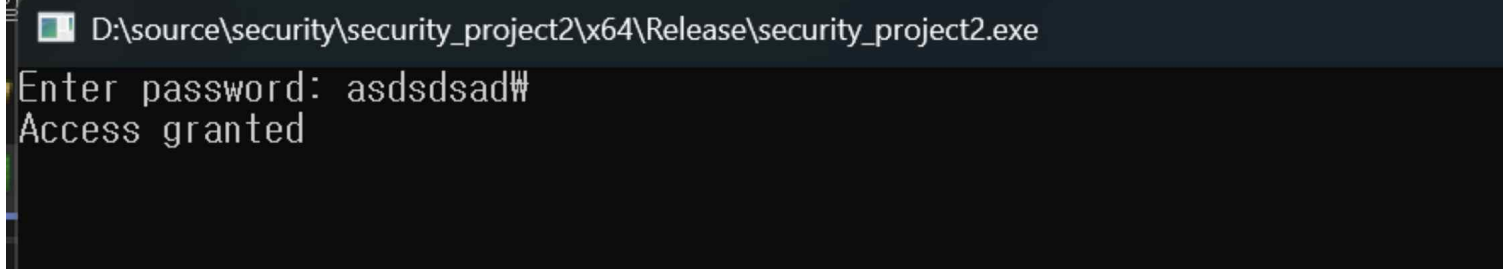


찾아낸 정답을 입력하니 정답이 나왔다

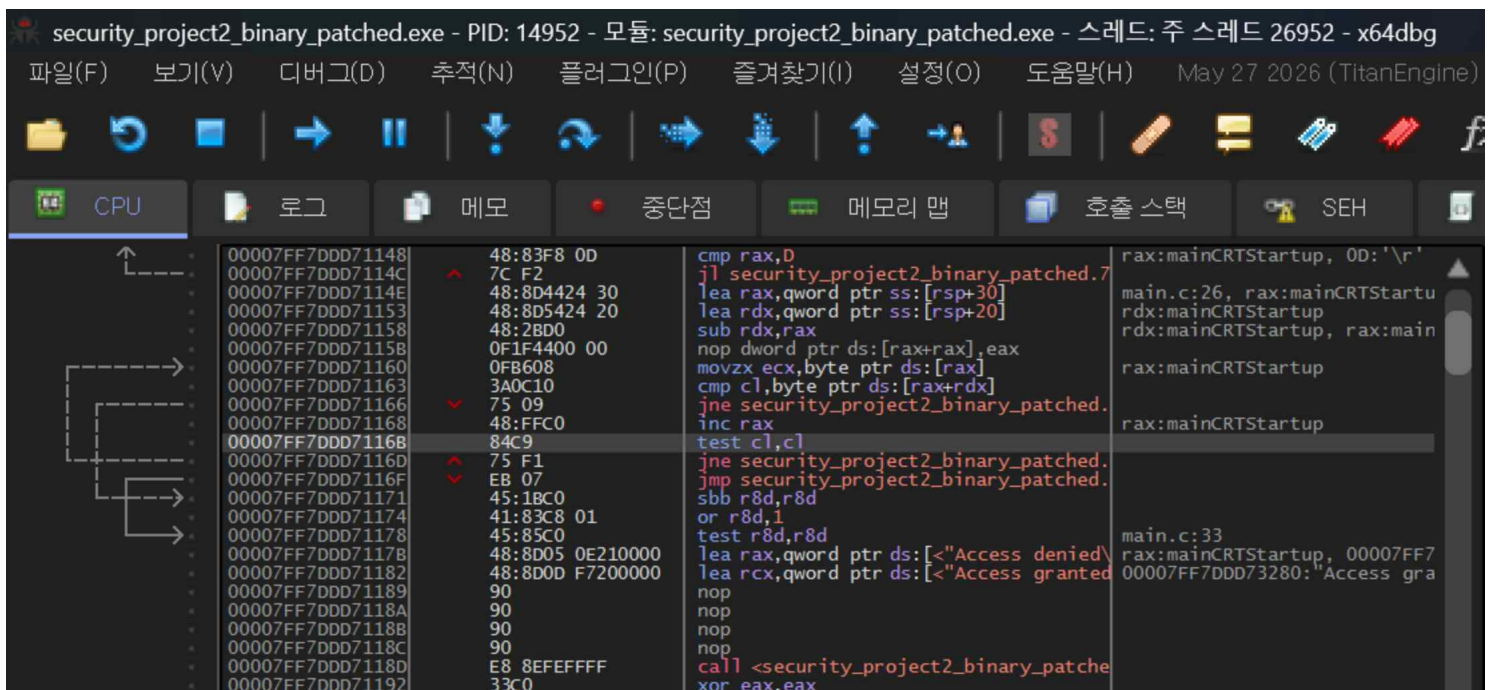
그 다음에는 이 실행파일에 바이너리 패치를 통해서 오답을 입력해도 정답이 되게 만들어 보겠다.



우선 ida에서 먼저 access granted를 결정짓는 핵심적인 구문 if(v3)의 주소를 얻어 xdbg64에서 그 위치로 가면 cmovne rcx, rax가 있는데 이 줄을 우클릭한 후에 바이너리를 누르고 fill with nop을 누르면 이 명령어가 사라지면서 잘못된 값을 입력해도 로그인이 된다. 이렇게 되는 이유는 cmovne rcx, rax가 ida에서 디컴파일된 부분의 if(v3): v8 = access denied가 되는 걸 아예 삭제하기 때문이고 이 외에도 다양한 바이너리 패치 기법 예를 들면 lea rax, ["Access denied"]를 lea rax, ["Access granted"]로 바꾸는 등의 방법으로 잘못된 입력을 넣어도 로그인을 할 수 있다.



물론 이는 다시 재실행되는 임시 버전이기 때문에 이를 영구적으로 만들기 위해서는 우클릭하고 패치를 누르고 반영할 패치 내용을 클릭한 후에 파일 패치를 누르고 그 다음에 새로운 이름으로 파일을 저장하면 된다.



위의 사진이 바로 영구패치가 된 버전의 실행파일이다.

배운 점

이런 기본적인 리버싱 프로젝트를 진행함에 따라 나는 정적 분석과 동적 분석이 어떻게 이루어지는지 그리고 전반적인 리버싱이 진행되는 과정에 대해서 알게 되었고 Ghidra,x64dbg,Detect It Easy,HxD,GCC등 다양한 리버싱 도구들을 사용하는 기초적인 방법에 대해서 알게 되었고 또한 바이너리 패치로 실제 해커들이 해킹할 수 있는 방법에 대해서도 알게 되었다.

한계점

하지만 내가 배운 내용은 매우 기초적인 내용이고 암호화나 해독이 너무 간단한 로그인 프로그램에 대해 분석했기 때문에 더 심화된 암호화가 잘 되고 훨씬 방대하고 큰 프로그램에 대한 분석과 연습이 필요하다. 이는 추후 다음 프로젝트를 통해서 진행할 예정이다.